Lesson plan · for the teacher

A timetable that works

Optimization · Level 3 · Lesson 1

Time
40 min

Materials
Board and chalk; Sticky notes or paper squares in four colours, one colour per subject; Optional: a phone with Darsi to run the model in the lab

Learning goals

- Write a timetable as a model: a period per lesson, and rules.
- See how a solver crosses out and searches, and what INFEASIBLE means.
- Judge a timetable that keeps every rule: is it also fair?

Activity: The sticky-note timetable

1. Warm-up (5 min): Ask pupils what goes wrong when a timetable is badly made: two lessons at once, a teacher in two rooms, a free period nobody wanted. List their answers as rules.
2. Teach (8 min): Draw a grid for three classes and four periods. Explain the two main rules (a class has one lesson at a time; a teacher is in one class at a time) and two extra rules: Ms Huda leaves after period 3, and 6A's maths test is in period 1.
3. Build it (15 min): Each group gets twelve sticky notes, four per class, coloured by subject. They place them on their own grid, then swap grids with another group to look for clashes.
4. New rule (7 min): Announce "the art room holds an exam in period 3". Groups move notes to keep every rule and count how many moved.
5. Wrap-up (5 min): Ask which class got the last period most often, and whether a timetable that keeps every rule is always fair. Show the lab or the modeller if you have a phone.

Prepare twelve notes per group: four lessons for each of 5A, 5B and 6A (maths, science, English, art), one colour per subject. No sticky notes? Squares of paper and a little tape work.

Discussion

- Which rule did we forget at first, and how did we notice?
- What does it mean when no timetable can keep every rule?
- Who got the last period, and is that fair?

Exit ticket

- Write one rule of our school's timetable, and one change that would make it fairer.

Try it at school tomorrow
Draw a timetable grid for three classes on the board, with sticky notes for the lessons. Pupils place them and call out every clash: a class with two lessons at once, or a teacher in two rooms. Then add one new rule, such as "the art room is busy in period 3", and count how many notes have to move.

Practice questions · for you

A timetable that works

Optimization · Level 3 · Lesson 1

1 **The solver found a timetable that keeps every rule, but 5B has maths in the last period every day, when pupils are tired. What do you do?**

- ◯ Go back to pencil and eraser
- ◯ Accept it: the solver says OPTIMAL
- ◯ Add a rule or a goal against it, and solve again

2 **Match each solver status to what it tells you.**

OPTIMAL	• • • •	Proved: no answer keeps all the rules
FEASIBLE	• • • •	Stopped before it was sure either way
INFEASIBLE	• • • •	The best answer (with no goal: an answer that keeps every rule)
UNKNOWN	• • • •	An answer that keeps every rule, maybe not the best

3 **Complete the timetable model: a class has one lesson at a time, each subject's teacher is in one class at a time, and Ms Huda (science) leaves after period 3.**

```
for c in classes:
    m. _____(
        [p[c, s] for s in subjects])
for s in subjects:
    m.add_all_different(
        [p[c, s] for c in _____])
for c in classes:
    m.add(p[c, "science"] _____ 3)
```

Word bank: `classes` `maximize` `add_all_different` `subjects` `>=` `<=`

4 **5A (a) and 5B (b) both have maths, science and English in periods 1 to 3. Each subject has one teacher for both classes, and Ms Huda (science) leaves after period 2. Add the missing rule: a teacher can't be in both classes in the same period. Then run it.**

```
from ortools.sat.python import cp_model
S = ["maths", "science", "english"]
m = cp_model.CpModel()
a = {s: m.new_int_var(1, 3, s) for s in S}
b = {s: m.new_int_var(1, 3, s) for s in S}
for d in (a, b):
    m.add_all_different(list(d.values()))
    m.add(d["science"] <= 2)
# a teacher: not in 5A and 5B at once

solver = cp_model.CpSolver()
print(solver.status_name(solver.solve(m)))
```

Write your program:

5 **If the solver says OPTIMAL for a timetable with no goal, the timetable is fair to every teacher and class.**

◯ True ◯ False

✎ **Which rule of your school's timetable is written down nowhere, but everyone knows? How would you write it as one line of a model?**

Check your answers

A timetable that works

Optimization · Level 3 · Lesson 1

1 **Add a rule or a goal against it, and solve again**

The solver keeps exactly the rules it is given, nothing more.

2 **OPTIMAL — The best answer (with no goal: an answer that keeps every rule)**

FEASIBLE — An answer that keeps every rule, maybe not the best

INFEASIBLE — Proved: no answer keeps all the rules

UNKNOWN — Stopped before it was sure either way

The status is the first thing to read. Only OPTIMAL and FEASIBLE come with a timetable; INFEASIBLE is a proof that the rules clash; UNKNOWN means the search ran out of time or nodes, so try a smaller model or more time.

3

```
for c in classes:
    m.add_all_different(
        [p[c, s] for s in subjects])
for s in subjects:
    m.add_all_different(
        [p[c, s] for c in classes])
for c in classes:
    m.add(p[c, "science"] <= 3)
```

add_all_different keeps a class's lessons in different periods; for each subject, the same teacher's lessons in every class must differ too; and p[c, "science"] <= 3 keeps science in periods 1 to 3, with the expression on the left.

4

```
from ortools.sat.python import cp_model
S = ["maths", "science", "english"]
m = cp_model.CpModel()
a = {s: m.new_int_var(1, 3, s) for s in S}
b = {s: m.new_int_var(1, 3, s) for s in S}
for d in (a, b):
    m.add_all_different(list(d.values()))
    m.add(d["science"] <= 2)
for s in S:
    m.add(a[s] != b[s])
solver = cp_model.CpSolver()
print(solver.status_name(solver.solve(m)))
```

Each subject has one teacher, so 5A's and 5B's periods for it must differ: one != rule per subject.

5 **False**

OPTIMAL only says every rule you wrote is kept.